Heaven's light is our guide

# Rajshahi University of Engineering & Technology

## Department of Computer Science & Engineering


# Project Report

**Course Name:** Operating Systems Sessional

**Course Code:** CSE 3202

**Project Name:** Bare metal kacchiOS design


**Submitted by:**

| Roll | Name |
|---|---|
| 2103064 | Md. Asad-Al-Adil |
| 2103067 | Md. Talha Bin Kobir |
| 2103068 | Mst. Maysha Khanom Moon |
| 2103069 | Dina Shanjida |
| 2103075 | Md. Arif Hossen |
| 2103078 | Fariha Anjum Aupy |

**Submitted to:**

**Md. Farhan Shakib**
Lecturer
Department of Computer Science and Engineering, RUET


**Date of Submission:** 25 Feb 2026

# Project Overview

In this project we are building the kacchiOS operating system that is a minimal and educational operating system. It has been created to function within a virtualized environment using QEMU. The main aim of the project is to gain experience with the essential mechanisms that allow an operating system to boot up, initialize interfaces to the hardware, and perform kernel-level code execution.

# Objectives

The major objectives of the kacchiOS project are as follows:

- To create a memory manager that handles the allocation and deallocation of memory for processes.
- To create a process manager that handles processes, their states, and their termination.
- To create a scheduler that handles context switching and a scheduling policy.
- To create a basic and functional operating system that shows the fundamental concepts of an operating system, such as processes and memory management.

# Project Components

## 1. Memory Manager

It allocates and de-allocates memory resources to all the processes executing within the system. It provides each of the processes which are executing their needed memory. The memory is de-allocated when the process is terminated.

### Key Features:

1. Heap Allocation:
   - Uses k_malloc to allocate memory from the heap by either reusing free blocks or extending the heap.
   - The function maintains a **free list** to manage free blocks and searches it using a first-fit strategy.

2. Coalescing Free Blocks:
   - When a block is freed with k_free, the memory manager checks if the next block in the list is also free. If so, it merges them to prevent fragmentation.

3. Memory Deallocation:
   - k_free marks a block as free and potentially coalesces adjacent free blocks into a larger free block, improving memory utilization.

### Code Snippets:

#### Memory Allocation (k_malloc):

```
void* k_malloc(uint32_t size) {
  BlockHeader* curr = free_list_head;

  while (curr) {
```

```
        if (curr->is_free && curr->size >= size) {
            curr->is_free = 0;
            return (void*)(curr + 1);  // Return memory after the header
        }
        curr = curr->next;
    }

    uint32_t total_size = sizeof(BlockHeader) + size;
    BlockHeader* new_block = (BlockHeader*)heap_top;
    heap_top += total_size;

    new_block->size = size;
    new_block->is_free = 0;
    new_block->next = 0;

    if (free_list_head == 0) {
        free_list_head = new_block;
    } else {
        BlockHeader* temp = free_list_head;
        while (temp->next != 0) {
            temp = temp->next;
        }
        temp->next = new_block;
    }
    return (void*)(new_block + 1);  // Return memory after the header
}
```

### Memory Deallocation (k_free):

```
void k_free(void* ptr) {
    if (ptr == 0) return;

    BlockHeader* header = (BlockHeader*)ptr - 1;
    header->is_free = 1;

    if (header->next != 0 && header->next->is_free == 1) {
        header->size += sizeof(BlockHeader) + header->next->size;
        header->next = header->next->next;  // Merge adjacent free blocks
    }
}
```

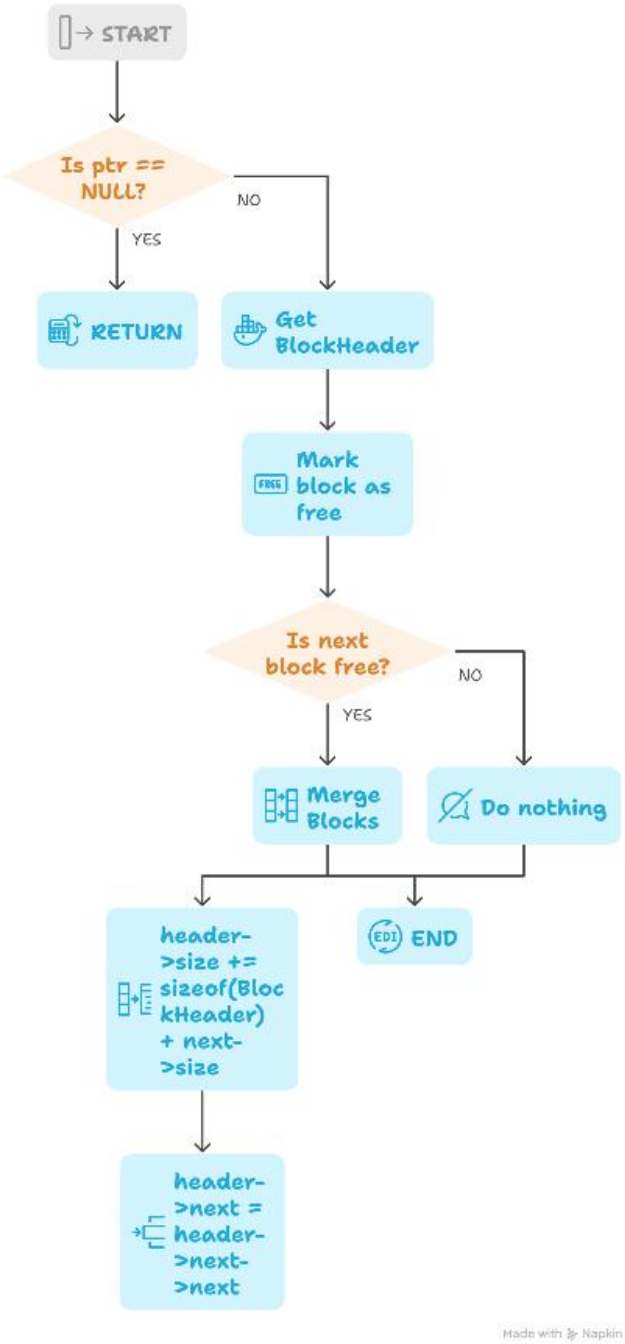



Fig: Memory deallocation and merging

## 2. Process Manager

It creates processes,does transition between process states, and then terminates processes when the process is done executing . It also manages the repository of processes through the process table, where they are represented by the Process Control Block (PCB), which contains process state, process identifier, and stack pointer.

## Key Features:

1. Process Creation (create_process):
   - Allocates memory for the process stack.
   - Sets up the stack with the entry point and a call to exit_process to terminate the process when done.
   - Adds the new process to the process table with an initial state of **READY**.

2. Process Termination (exit_process):
   - Marks the process as **TERMINATED**, frees its allocated stack memory, and triggers the scheduler to pick the next process.

3. Process Table:
   - Stores the information for each process, including the **PID**, **state** (READY, CURRENT, TERMINATED), and **stack pointer**.

4. Process States:
   - READY: The process is ready to run.
   - CURRENT: The process is currently running.
   - TERMINATED: The process has finished execution.

## Code Snippets:

### Process Creation (create_process):

```
int create_process(void (*entry_point)()) {
   if (process_count >= MAX_PROCESSES)
      return -1;

   int idx = -1;
   for (int i = 0; i < MAX_PROCESSES; i++) {
      if (process_table[i].state == TERMINATED) {
         idx = i;
         break;
      }
   }
   if (idx == -1)
      return -1;
   void *stack = k_malloc(STACK_SIZE);

   serial_puts("[Process Manager] Creating PID ");
   print_dec(idx + 1);
   serial_puts("...\n");

   uint32_t *sp = (uint32_t *)((uint8_t *)stack + STACK_SIZE);
   *(--sp) = (uint32_t)exit_process;
   *(--sp) = (uint32_t)entry_point;
```

```
    process_table[idx].pid = idx + 1;
    process_table[idx].state = READY;
    process_table[idx].esp = sp;
    process_table[idx].stack_start = stack;
    process_count++;
    return idx;
}
```

### Process Termination (exit_process):

```
void exit_process() {
    serial_puts("[Process Manager] PID ");
    print_dec(process_table[current_process_index].pid);
    serial_puts(" Terminated. (State: TERMINATED)\n");
    process_table[current_process_index].state = TERMINATED;
    serial_puts("   -> Freeing Stack Memory...\n");
    k_free(process_table[current_process_index].stack_start);
    schedule();
}
```

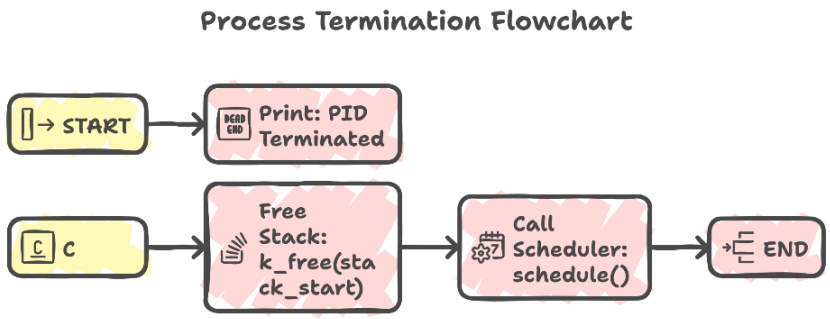



Fig: Flowchart of exit_process()

## 3. Scheduler

The scheduler controls the order in which the processes are executed. It implements a round-robin scheduling algorithm and performs context switches on the processes. The scheduler ensures that each process gets some time on the CPU, which can be achieved by using an aging technique.

### Key Features:

1. Process State Transitions:

- When a process is currently running, the scheduler changes its state from CURRENT to READY.
- The next process is selected from the READY state and is moved to the CURRENT state.

2. Context Switching:

- The context_switch function is responsible for saving and restoring the process states during a switch, ensuring smooth transitions between processes.

3. Round-Robin Algorithm:

- The scheduler performs a round-robin traversal of the process table, selecting the next process in the queue when it's ready.

## Code Snippet:

### Scheduler (schedule):

```
void schedule() {
   int next_idx = -1;
   int start_idx = (current_process_index == -1) ? 0 : (current_process_index + 1);
   serial_puts("\n[Scheduler] Context Switch:\n");

   PCB* prev_proc = 0;
   if (current_process_index != -1 && process_table[current_process_index].state == CURRENT) {
      serial_puts("   PID ");
      print_dec(process_table[current_process_index].pid);
      serial_puts(" (CURRENT) -> READY\n");

      process_table[current_process_index].state = READY;
      prev_proc = &process_table[current_process_index];
   }

   for (int i = 0; i < MAX_PROCESSES; i++) {
      int idx = (start_idx + i) % MAX_PROCESSES;

      if (process_table[idx].state == TERMINATED && process_table[idx].pid != 0) {
         serial_puts("   PID ");
         print_dec(process_table[idx].pid);
         serial_puts(" (TERMINATED) -> Skipping\n");
      }
      else if (process_table[idx].state == READY) {
         next_idx = idx;
         break;
      }
   }
   if (next_idx == -1) return;

   serial_puts("   PID ");
   print_dec(process_table[next_idx].pid);
   serial_puts(" (READY)   -> CURRENT\n");

   current_process_index = next_idx;
   PCB* next_proc = &process_table[next_idx];
   next_proc->state = CURRENT;

   uint32_t* dummy_sp;
```

```
    uint32_t** old_sp_ptr = (prev_proc) ? &prev_proc->esp : &dummy_sp;
    context_switch(old_sp_ptr, next_proc->esp);
}
```

**Context Switching (context_switch):**

```
__attribute__((naked)) void context_switch(uint32_t** old_sp, uint32_t* new_sp) {
    (void)old_sp; (void)new_sp;
    __asm__ volatile (
        "pusha\n\t"
        "pushf\n\t"
        "mov 40(%esp), %eax\n\t"
        "mov %esp, (%eax)\n\t"
        "mov 44(%esp), %esp\n\t"
        "popf\n\t"
        "popa\n\t"
        "ret\n\t"
    );
}
```

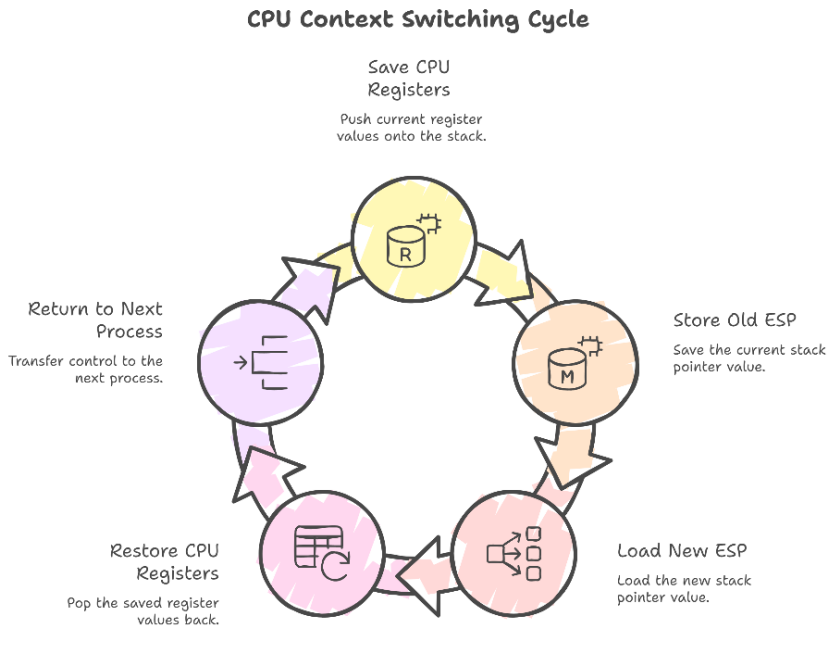
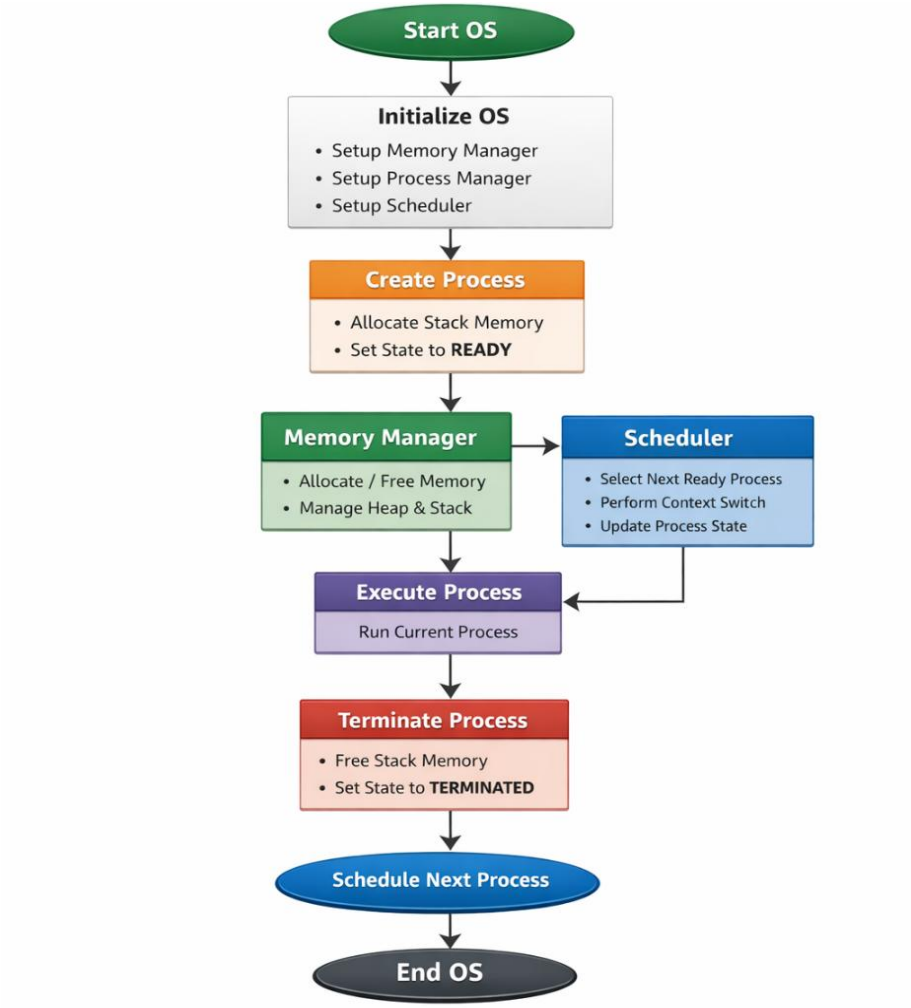



Fig: Flowchart of context_switch()

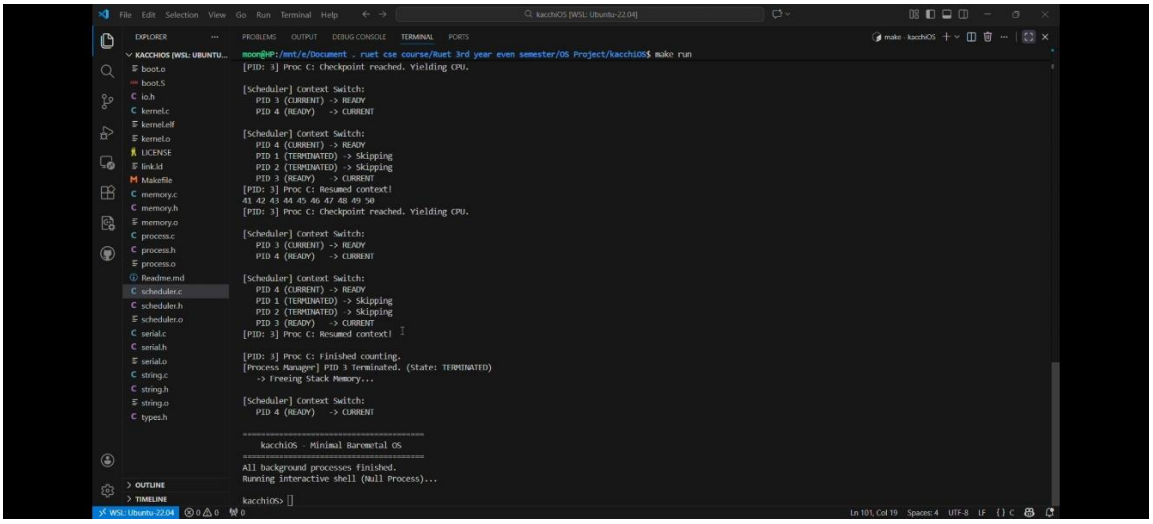
## KacchiOS Architecture

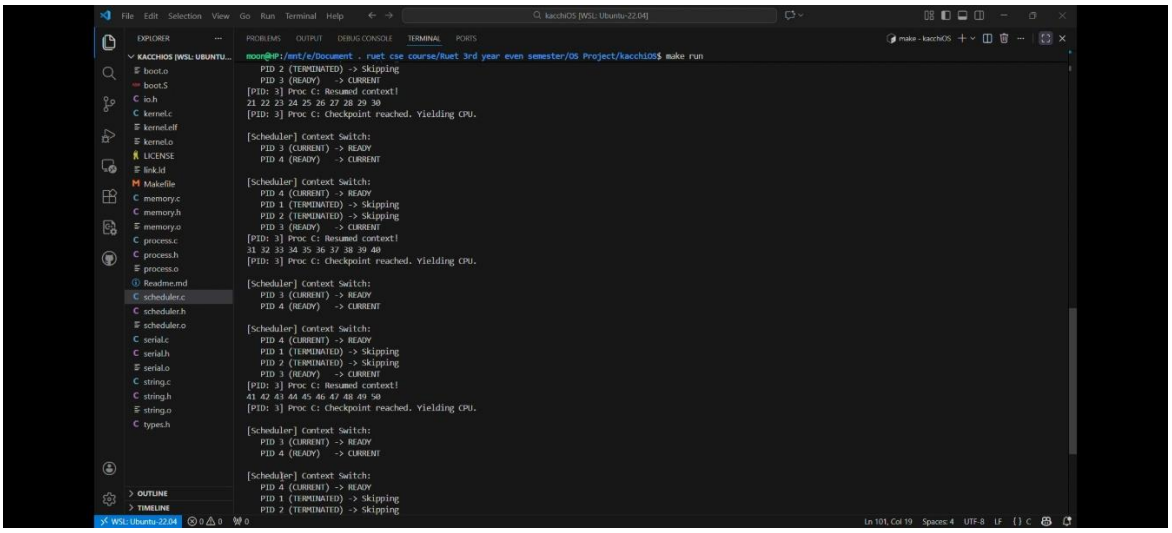
kacchiOS is created in three major components which include Memory Manager to allocate memory, Process Manager to deal with processes and the Scheduler to deal with CPU time. The combination of them makes different tasks flow and the processes efficient.

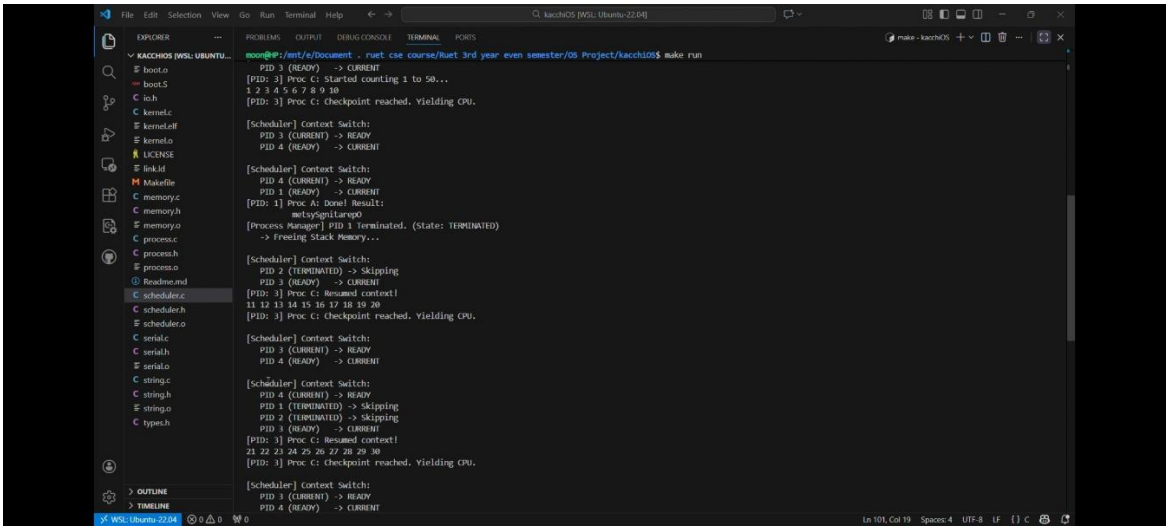


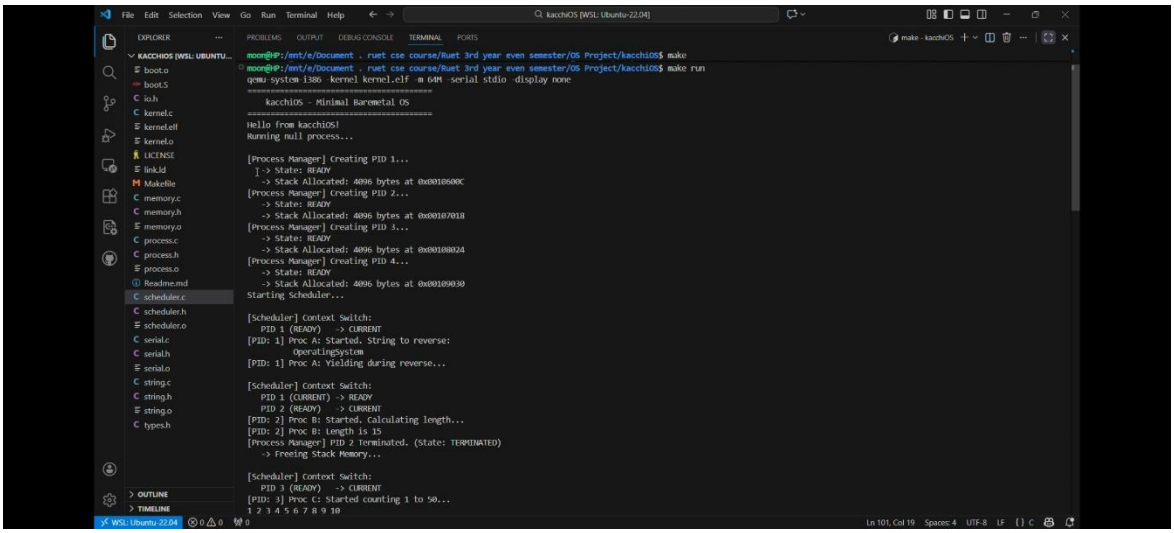

## Output Screenshots

The following screenshots show the kacchiOS system in action:




moon@HP:/mnt/e/Document . ruet cse course/Ruet 3rd year even semester/OS Project/kacchiOS$ make run
PID 2 (TERMINATED) -> Skipping
PID 3 (READY) -> CURRENT
[PID: 3] Proc C: Resumed context!
21 22 23 24 25 26 27 28 29 30
[PID: 3] Proc C: Checkpoint reached. Yielding CPU.
[Scheduler] Context Switch:
PID 3 (CURRENT) -> READY
PID 4 (READY) -> CURRENT
[Scheduler] Context Switch:
PID 4 (CURRENT) -> READY
PID 1 (TERMINATED) -> Skipping
PID 2 (TERMINATED) -> Skipping
PID 3 (READY) -> CURRENT
[PID: 3] Proc C: Resumed context!
31 32 33 34 35 36 37 38 39 40
[PID: 3] Proc C: Checkpoint reached. Yielding CPU.
[Scheduler] Context Switch:
PID 3 (CURRENT) -> READY
PID 4 (READY) -> CURRENT
[Scheduler] Context Switch:
PID 4 (CURRENT) -> READY
PID 1 (TERMINATED) -> Skipping
PID 2 (TERMINATED) -> Skipping
PID 3 (READY) -> CURRENT
[PID: 3] Proc C: Resumed context!
41 42 43 44 45 46 47 48 49 50
[PID: 3] Proc C: Checkpoint reached. Yielding CPU.
[Scheduler] Context Switch:
PID 3 (CURRENT) -> READY
PID 4 (READY) -> CURRENT
[Scheduler] Context Switch:
PID 4 (CURRENT) -> READY
PID 1 (TERMINATED) -> Skipping
PID 2 (TERMINATED) -> Skipping


moon@HP:/mnt/e/Document . ruet cse course/Ruet 3rd year even semester/OS Project/kacchiOS$ make run
PID 3 (READY) -> CURRENT
[PID: 3] Proc C: Started counting 1 to 50...
1 2 3 4 5 6 7 8 9 10
[PID: 3] Proc C: Checkpoint reached. Yielding CPU.
[Scheduler] Context Switch:
PID 3 (CURRENT) -> READY
PID 4 (READY) -> CURRENT
[Scheduler] Context Switch:
PID 4 (CURRENT) -> READY
PID 1 (READY) -> CURRENT
[PID: 1] Proc A: Done! Result:
motsySgnilarepO
[Process Manager] PID 1 Terminated. (State: TERMINATED)
-> Freeing Stack Memory...
[Scheduler] Context Switch:
PID 2 (TERMINATED) -> Skipping
PID 3 (READY) -> CURRENT
[PID: 3] Proc C: Resumed context!
11 12 13 14 15 16 17 18 19 20
[PID: 3] Proc C: Checkpoint reached. Yielding CPU.
[Scheduler] Context Switch:
PID 3 (CURRENT) -> READY
PID 4 (READY) -> CURRENT
[Scheduler] Context Switch:
PID 4 (CURRENT) -> READY
PID 1 (TERMINATED) -> Skipping
PID 2 (TERMINATED) -> Skipping
PID 3 (READY) -> CURRENT
[PID: 3] Proc C: Resumed context!
21 22 23 24 25 26 27 28 29 30
[PID: 3] Proc C: Checkpoint reached. Yielding CPU.
[Scheduler] Context Switch:
PID 3 (CURRENT) -> READY
PID 4 (READY) -> CURRENT


moon@HP:/mnt/e/Document . ruet cse course/Ruet 3rd year even semester/OS Project/kacchiOS$ make
moon@HP:/mnt/e/Document . ruet cse course/Ruet 3rd year even semester/OS Project/kacchiOS$ make run
qemu-system-i386 -kernel kernel.elf -m 64M -serial stdio -display none
kacchiOS - Minimal Baremetal OS
Hello from kacchiOS!
Running null process...
[Process Manager] Creating PID 1...
-> State: READY
-> Stack Allocated: 4096 bytes at 0x0010600C
[Process Manager] Creating PID 2...
-> State: READY
-> Stack Allocated: 4096 bytes at 0x00107018
[Process Manager] Creating PID 3...
-> State: READY
-> Stack Allocated: 4096 bytes at 0x00108024
[Process Manager] Creating PID 4...
-> State: READY
-> Stack Allocated: 4096 bytes at 0x00109030
Starting Scheduler...
[Scheduler] Context Switch:
PID 1 (READY) -> CURRENT
[PID: 1] Proc A: Started. String to reverse:
OperatingSystem
[PID: 1] Proc A: Yielding during reverse...
[Scheduler] Context Switch:
PID 1 (CURRENT) -> READY
PID 2 (READY) -> CURRENT
[PID: 2] Proc B: Started. Calculating length...
[PID: 2] Proc B: Length is 15
[Process Manager] PID 2 Terminated. (State: TERMINATED)
-> Freeing Stack Memory...
[Scheduler] Context Switch:
PID 3 (READY) -> CURRENT
[PID: 3] Proc C: Started counting 1 to 50...
1 2 3 4 5 6 7 8 9 10

# Challenges & Solutions

## Challenges:

**1. Memory Fragmentation:**
Efficient memory allocation was a challenge, especially managing heap memory. The buddy system was implemented to minimize fragmentation.

**2. Context Switching Efficiency:**
Context switching can be computationally expensive. To minimize the overhead, we optimized the state-saving and restoring process and reduced the time quantum.

**3. Process Starvation:**
To prevent process starvation in the round-robin scheduler, we implemented an aging mechanism that gradually increases the priority of processes waiting too long.

## Solutions:

1. The **buddy system** helped reduce memory fragmentation by merging adjacent free blocks of memory into larger blocks.

2. **Optimized context switching** by saving only the essential state information, reducing the overhead.

3. **Aging** in the scheduler ensured that processes would not starve by gradually increasing their priority if they had been waiting for too long.

# Conclusion

This kacchiOS project has all of the essential components of an operating system. This project demonstrates memory allocation mechanisms, process management, and process scheduling in an operating system.

# References

https://github.com/Maysha-Khanom-Moon/Series_21_Section_B_Group_B1_kacchiOS